

## TRABALHO PRÁTICO

- O projeto deve ser submetido num ficheiro compactado (ZIP) ou através do repositório indicado ao formador;
- A cotação final será o somatório das cotações parciais de cada funcionalidade implementada corretamente, conforme os critérios de avaliação.

### Sistema "FastBite"

O objetivo é desenvolver um ecossistema de 3 APIs em ASP.NET Core que permita a gestão e venda de refeições para um serviço de *delivery*

#### 1. Arquitetura e Fluxo de Dados

- **Identity.API:** O ponto de entrada. Autentica e autoriza.
- **MenuCatalog.API:** O inventário. Diz o que existe no menu e quanto custa.
- **DeliveryOrdering.API:** O processo. Cria o pedido de entrega e confirma o pagamento.

#### 2. Requisitos Funcionais (Backlog)

##### API 1: Identity.API (Segurança Centralizada)

- **RF1:** Registo de novos utilizadores com ASP.NET Core Identity.
- **RF2:** Login com emissão de Token JWT contendo: sub (ID do utilizador), email e role (Admin/Restaurante ou Cliente).
- **RF3:** Implementação de uma política de "Password Strong" (mínimo 8 caracteres, números e símbolos).

##### API 2: MenuCatalog.API (Gestão de Conteúdo)

- **RF4:** CRUD completo de Pratos/Menus (Nome, Descrição, Categoria, Alergénios, Preço Base, Limite Diário de Doses).
- **RF5:** Endpoint público (sem token) para listagem de pratos disponíveis.
- **RF6:** Endpoint privado (apenas Admin) para cancelamento/remoção de pratos do menu.
- **RF7:** Endpoint de "CheckAvailability": Recebe idPrato e quantidade e retorna true/false.

##### API 3: DeliveryOrdering.API (Venda e Reserva)

- **RF8:** Endpoint POST /order (equivalente ao /buy):
  1. Valida se o utilizador está autenticado.
  2. Consulta a MenuCatalog.API (via HttpClient) para verificar se há doses disponíveis.
  3. Calcula o valor total.
  4. Guarda o pedido na BD com estado "Pendente".
- **RF9:** Histórico de Compras: O utilizador autenticado pode ver apenas os seus pedidos passados.

### 3. Requisitos Técnicos (O teu "Standard" de Qualidade)

Para obteres a nota máxima, deves aplicar estes conceitos:

- **Padrão Repository:** Isolar o acesso a dados (Entity Framework Core) da lógica de negócio.
- **DTOs & AutoMapper:** Não expor as entidades da base de dados. Criar modelos específicos para Request e Response.
- **Dependency Injection:** Configurar corretamente o tempo de vida dos serviços (Transient, Scoped, Singleton).
- **Validations:** Usar Data Annotations ou FluentValidation para impedir dados inválidos (ex: datas no passado).
- **Configuração Centralizada:** Usar o appsettings.json para guardar as URLs das outras APIs e a chave secreta do JWT.

#### Guia de Implementação (Passo a Passo) [Exemplo]

1. **Semana 1:** Cria a Solução e o projeto **Identity.API**. Garante que consegues gerar um Token válido no Swagger.
2. **Semana 2:** Cria a **Catalog.API**. Implementa o CRUD de eventos e testa a persistência no SQL Server.
3. **Semana 3:** Cria a **Ordering.API**. Aqui vais precisar de usar o `IHttpClientFactory` para ligar as pontas.
4. **Semana 4:** Refatorização, tratamento de erros e documentação final no Swagger.

#### Boas Práticas Obrigatórias

Para que o projeto seja considerado de nível profissional, deves aplicar:

- **Separação de Camadas:** Nada de lógica de negócio dentro dos Controllers. Utiliza **Services** e **Repositories**.
- **Data Transfer Objects (DTOs):** Nunca exponhas as tuas entidades da base de dados diretamente no JSON de resposta.
- **Documentação:** Todas as APIs devem ter o **Swagger** configurado e organizado.
- **Tratamento de Erros:** Implementação de um Middleware global para capturar exceções e retornar respostas amigáveis (ex: `ProblemDetails`).

### Exemplo de Estrutura de Comunicação

Exemplo de como a Ordering API deve consultar a Catalog API usando IHttpClientFactory:

```
// Exemplo de um serviço na Ordering API que consulta o Catálogo
public class CatalogService : ICatalogService
{
    private readonly HttpClient _httpClient;

    public CatalogService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<bool> IsEventAvailable(Guid eventId, int quantity)
    {
        // Chamada à API de Catálogo
        var response = await _httpClient.GetAsync($"api/events/{eventId}/availability?quantity={quantity}");

        if (!response.IsSuccessStatusCode)
            return false;

        var result = await response.Content.ReadFromJsonAsync<AvailabilityResponse>();
        return result.IsAvailable;
    }
}
```

## Critérios de Avaliação

### 1. Estrutura e Organização (4 Valores)

- **Separação de Preocupações:** Uso de *Controllers* apenas para rotas e *Services* para a lógica de negócio.
- **Clean Code:** Nomes de variáveis e métodos em inglês (padrão de indústria) ou português consistente, seguindo as *naming conventions* da Microsoft (PascalCase para métodos/classes).
- **Injeção de Dependência:** Uso correto do *IHttpClientFactory* e de serviços registados no *Program.cs*.
- **Swagger/OpenAPI:** Todos os endpoints devem estar documentados, com resumos claros do que fazem.

### 2. Gestão de Dados e EF Core (4 Valores)

- **Migrations:** Base de dados gerada e mantida através de *Entity Framework Migrations*.
- **DTOs (Data Transfer Objects):** As entidades da base de dados (ex: Evento) **nunca** devem ser devolvidas diretamente. Deves usar DTOs para entrada e saída de dados.
- **Relacionamentos:** Configuração correta de relações 1:N (ex: Um Evento tem muitos Bilhetes/Reservas).

### 3. Implementação da Identity API (4 Valores)

- **JWT funcional:** Capacidade de registar um utilizador, fazer login e obter um token válido.
- **Claims:** O token deve conter, pelo menos, o *UserId* e o *Role*.
- **Segurança:** Password hashing (nativo do ASP.NET Identity) e não guardar passwords em plain text.

### 4. Lógica de Interoperabilidade (3 Valores)

- **Comunicação entre APIs:** A *Ordering.API* consegue fazer um GET à *Catalog.API* para validar stock com sucesso.
- **Tratamento de Status Codes:** Uso correto de 200 OK, 201 Created, 400 BadRequest, 404 NotFound e 401 Unauthorized.
- **Resiliência Básica:** Verificação de que a API de destino está ativa antes de tentar processar a compra.